Software Architecture

SE6103

Sachin Tharaka
(BSc Honours in Software Engineering)

SE6103 – Course Content

- Topic 01: Basic concepts & principles about software architecture
- Topic 02: Introduction to Software Architectural pattern
- Topic 03: ADL
- Topic 04: 4+1 Architecture
- Topic 05: Practical approaches & methods for Create & Analyse software architecture
- Topic 06: Quality attributes of software architectures
- Topic 07: Examples in architectural design applications & case studies in software architecture (N tier architecture, SOA, Cloud, etc.)

SE6103 – Course Content

- Topic 01: Basic concepts & principles about software architecture
- Topic 02: **Introduction to Software Architectural pattern**
- Topic 03: ADL
- Topic 04: 4+1 Architecture
- Topic 05: Practical approaches & methods for Create & Analyse software architecture
- Topic 06: Quality attributes of software architectures
- Topic 07: Examples in architectural design applications & case studies in software architecture (N tier architecture, SOA, Cloud, etc.)

Introduction to Software Architectural pattern

Software Architectural Patterns

Software Architecture focuses on

- High-level system structure
- Components and connectors
- System quality attributes
- Architectural decisions

Key question architects ask,
How should we structure the system?

Typical Architecture?

What architecture does a typical web application use?

The Problem Architects Face

When building large software systems, engineers must decide:

- How components interact
- How data flows
- How responsibilities are divided
- How the system scales

Without structure systems become

- Hard to maintain
- Hard to scale
- Difficult to debug

Early software systems often became **spaghetti** architecture.
Architectural patterns provide **proven** solutions.

Architectural Pattern?

A Software Architectural Pattern is a reusable solution to a commonly occurring problem in software architecture.

- System structure
- Component relationships
- Communication mechanisms
- Design constraints

Patterns represent best practices learned over time.

Architectural Pattern vs Design Pattern

Pattern	Focus	Example
Architectural	• High level structure • Defines system architecture	• Layered architecture • Microservices
Design	• Lower-level solution • Solves component-level problems	• Singleton • Factory • Observer

Architecture patterns shape the whole system.

Pattern vs Architectural Style

Architecture	Focus
Style	Broad category of architecture
Pattern	Concrete implementation approach

- Style: Layered Architecture
  - Pattern: MVC

- Style: Service-based
  - Pattern: Microservices

Patterns are often implementations of styles.

Why Architectural Patterns Are Important

Architectural patterns provide

- Proven solutions
- Faster design decisions
- Better communication among developers
- Standardization across systems
- Improved maintainability

Most enterprise systems use Layered architecture, Why?.

Benefits of Using Patterns

- Reduced development risk
- Improved scalability
- Improved maintainability
- Better modularity
- Better team collaboration
- Organizations reuse architecture knowledge.

Consequences of Bad Architecture

Without proper architecture: Systems become tightly coupled.

- Difficult maintenance
- Poor performance
- Scalability limitations
- Increased development cost

Early monolithic enterprise systems.

Categories of Architectural Patterns

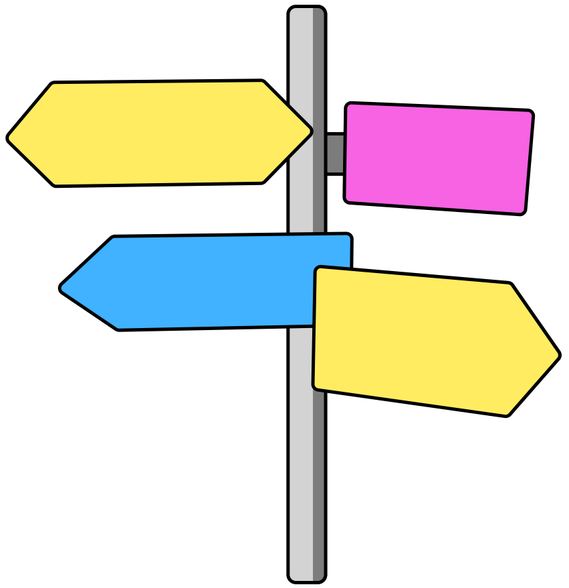
- Layered Architecture
- Client-Server Architecture
- Microservices Architecture
- Event-Driven Architecture
- Service-Oriented Architecture
- Pipe and Filter

Each solves different architectural problems.

Layered Architecture

The system is divided into layers.

- Presentation Layer
- Application Layer
- Business Logic Layer
- Data Layer



Each layer only communicates with adjacent layers.

Examples: Web applications, Enterprise systems

Example of Layered Architecture

Online Shopping System

Presentation	Application	Business Layer	Data Layer
Web interface	Request handling	Payment validation	Database access

Clear separation of responsibilities

Client–Server Architecture

One of the earliest architectural patterns

Client

- Sends requests

Server

- Processes requests
- Returns responses

Examples: Web applications, Email systems, Database servers

Microservices Architecture

Modern architectural pattern, System divided into small independent services.

Each service

- Performs a specific business function
- Runs independently
- Can be deployed independently

Example services: User service, Payment service, Order service, Notification service

Event-Driven Architecture

Architecture built around events. Components communicate via events.

Example events: User registered, Payment completed, Order shipped

Components react to events asynchronously.

- IoT systems
- Real-time systems

Service-Oriented Architecture (SOA)

SOA organizes software into services. Services communicate using standard protocols.

Characteristics

- Loose coupling
- Reusability
- Interoperability

Example: Enterprise integration systems.

Pipe and Filter Architecture

System processes data as a sequence of transformations. Components called **filters**, Data flows through **pipes**.

Example: Compiler architecture

Also used in: Data processing pipelines.

Choosing an Architectural Pattern

Choosing the correct architecture depends on

- System size
- Scalability requirements
- Performance requirements
- Team structure
- Business goals
- No single pattern works for every system.
- Architects often combine patterns.

Combining Multiple Patterns

Modern systems use hybrid architectures. in an E-commerce system

- Layered architecture internally
- Microservices for large modules
- Event-driven communication
- Hybrid architecture is common.

Architecture Trade-offs

Microservices

Advantages

- Scalability
- Flexibility

Disadvantages

- Complexity
- Network overhead

Architects must balance quality attributes.

Case Study

Design architecture for a **Food Delivery System**

Identify

- Major components
- Communication style
- Possible architectural pattern

Key Takeaways

Architectural patterns

- Provide reusable architecture solutions
- Help structure large systems
- Improve maintainability and scalability

Common patterns include

- Layered
- Client–Server
- Microservices
- Event-driven
- SOA

Architecture vs Implementation Thinking

“If the code works, the system is good.”

- Reality: System may Fail under load, Be insecure, Be impossible to maintain
- Architecture thinking is long-term thinking

Course Roadmap

- Fundamentals
- Architectural patterns
- ADL
- 4+1 model
- Practical design approaches
- Quality attributes
- Case studies (N-tier, SOA, Cloud)

Activity

- Why did Facebook move from monolithic PHP to a distributed architecture?
- What happens if a banking system has poor availability?
- Which is more important: performance or security?

Key Takeaways

- Software architecture defines system structure
- Influences quality attributes
- Impacts business success
- Is difficult to change
- Must be documented and evaluated

Introduction to Software Architectural patterns

Thank You!